

Tracé Incrémental Discret de Droites & Optimisation Entière

BUT2 Informatique Arles • Département Informatique • M. Éric Rémy

- **Le problème fondamental du tracé vectoriel** : Comment relier deux points (x_1, y_1) et (x_2, y_2) sur une grille discrète de pixels carrés de façon continue (8-connexité) et visuellement optimale ?
- **L'invention de Jack Bresenham (IBM, 1962)** : Conçu à l'origine pour piloter des traceurs incrémentaux CalComp. C'est le standard universel de la rasterisation 2D/3D moderne.
- **Objectif clé de performance** : Remplacer les équations continues coûteuses ($y = m \cdot x + p$) impliquant divisions et nombres flottants par des *additions entières et comparaisons de signe* ultra-rapides.

POURQUOI BRESENHAM PLUTÔT QUE L'ÉQUATION CONTINUE ?

L'algorithme naïf calcule pour chaque x : $y = \text{round}(m \cdot x + p)$.

- Il nécessite une division initiale ($\Delta y / \Delta x$) et une multiplication flottante par pixel.
- Sur le matériel graphique ou les processeurs embarqués, l'arithmétique flottante ralentit considérablement la cadence d'affichage.
- **Bresenham entier** garantit une cadence maximale avec **0 division** et **0 calcul flottant** dans la boucle !

Spécifications du TP5 :

1. Implantation dans le premier octant ($0 \leq \Delta y \leq \Delta x$).
2. Extension aux deux premiers octants (gestion de la pente > 1).
3. Généralisation aux 8 octants du plan (faisceau radial 360°).

Théorie Mathématique : Formulation du 1er Octant

- **Hypothèse du 1er Octant** : $x_1 \leq x_2$ et $0 \leq (y_2 - y_1) \leq (x_2 - x_1)$.
La pente $m = \Delta y / \Delta x \in [0, 1]$. À chaque pas, x s'incrmente de 1 ($x_{i+1} = x_i + 1$).

- **Dilemme discret au point $x_i + 1$** : Choisir entre :
 - Le pixel Horizontal **H** : $(x_i + 1, y_i)$
 - Le pixel Diagonal **D** : $(x_i + 1, y_i + 1)$

Distances au segment réel $y = m(x_i + 1) + p$:

$$d_1 = y_{\text{réel}} - y_i = m(x_i + 1) + p - y_i$$

$$d_2 = (y_i + 1) - y_{\text{réel}} = y_i + 1 - [m(x_i + 1) + p]$$

$$d_1 - d_2 = 2m(x_i + 1) + 2p - 2y_i - 1 = 2(\Delta y / \Delta x)(x_i + 1) + 2p - 2y_i - 1$$

POURQUOI L'ASTUCE DU CRITÈRE MULTIPLIÉ PAR Δx ?

La différence $d_1 - d_2$ contient la fraction $\Delta y / \Delta x$.

Puisque $\Delta x > 0$, son signe est strictement identique au signe de :

$$e_i = \Delta x \cdot (d_1 - d_2) \text{ (valeur 100\% ENTIÈRE)}$$

- Si $e_i < 0$: $d_1 < d_2 \Rightarrow$ le pixel H (y_i) est le plus proche.
- Si $e_i \geq 0$: $d_1 \geq d_2 \Rightarrow$ le pixel D ($y_i + 1$) est le plus proche !



Dérivation Incrémentale : Les Constantes c_1 et c_2

● Calcul récurrent du critère $e_{i+1} - e_i$:

Pour passer de l'étape i à l'étape $i+1$, on étudie la variation de e sans réévaluer l'équation continue :

Cas 1 : $e_i < 0$ (choix Horizontal $y_{i+1} = y_i$) :

$$e_{i+1} - e_i = 2 \Delta y \implies e_{i+1} = e_i + c_2 \text{ où } c_2 = 2 \Delta y$$

Cas 2 : $e_i \geq 0$ (choix Diagonal $y_{i+1} = y_i + 1$) :

$$e_{i+1} - e_i = 2 \Delta y - 2 \Delta x \implies e_{i+1} = e_i + c_1 \text{ où } c_1 = c_2 - 2 \Delta x$$

Initialisation en (x_1, y_1) :

Pour le premier point d'évaluation à $x_1 + 1$, le critère initial vaut :

$$\text{critère}_0 = 2 \Delta y - \Delta x = c_2 - \Delta x$$

POURQUOI CET ALGORITHME EST-IL DIT "INCRÉMENTAL" ?

Toutes les constantes (c_1 , c_2 , critère₀) sont pré-calculées **une seule fois** avant la boucle.

Dans le corps de la boucle, le passage au pixel suivant ne demande que :

- **1 test de signe** : `if (critere >= 0)`
- **1 addition entière** : `critere += c1` ou `critere += c2`
- **1 incrémentation** : `x++` (et optionnellement `y++`).

Complexité : **$O(\Delta x)$** opérations élémentaires à la cadence registre du processeur.

```
// Pseudo-code canonique de M. Eric Remy (TP5)
const int c2 = 2 * (y2 - y1);
const int c1 = c2 - 2 * (x2 - x1);
int critere = c2 - (x2 - x1);
while (x <= x2) {
    pixel(x, y) = couleur;
    if (critere >= 0) { y++; critere += c1; }
    else { critere += c2; }
    x++;
}
```

Question 1 : Implantation & Test du Premier Octant

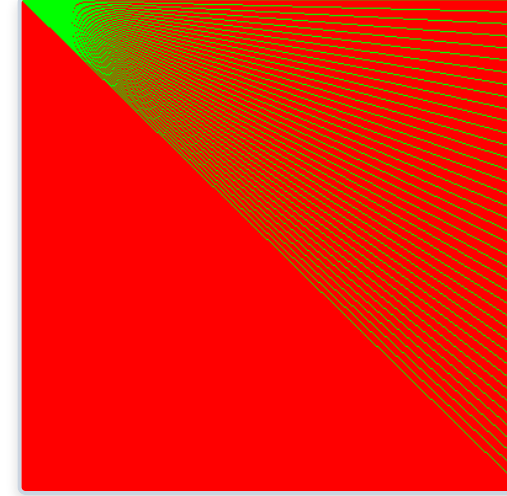
- **Méthode membre** : `ColorImage::lineOctant1(x1, y1, x2, y2, couleur)`.
- **Protocole de test imposé** :
 - Image 400x400 sur fond rouge uni `Color(255, 0, 0)`.
 - Faisceau de lignes vertes `Color(0, 255, 0)` partant toutes de (0,0).
 - Extrémités (399, y) espacées de 10 pixels sur le bord droit (y = 0, 10, ..., 390).

POURQUOI LE TRACÉ S'ARRÊTE-T-IL À LA DIAGONALE ?

Sur le bord droit, $x = 399$. La coordonnée y varie de 0 à 399.

Tant que $y \leq x$, la condition $0 \leq \Delta y \leq \Delta x$ est respectée : on reste strictement dans le **1er octant** (secteur angulaire de 0° à 45°).

Si $y > x$, la pente dépasse 1 : des "trous" apparaîtraient si l'on continuait d'avancer en x .



tp5_test1.ppm (400 × 400)

Faisceau du 1er octant généré par notre code C++

Question 2 : Extension au 2ème Octant (Pente > 1)

- **Problématique de l'Octant 2 ($0 \leq \Delta x < \Delta y$) :**

La droite monte plus vite qu'elle n'avance (pente > 1). Faire varier x d'un pas conduirait à un saut de plusieurs pixels en y, brisant la 8-connexité.

- **L'inversion des rôles de x et y :**

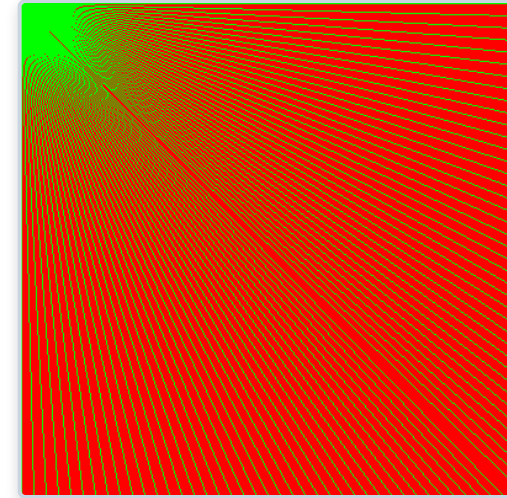
Il suffit de faire varier la variable y (axe prépondérant) et de calculer le décalage sur x :

- $c_2 = 2 \cdot \Delta x$ et $c_1 = c_2 - 2 \cdot \Delta y$
- $\text{critère}_0 = c_2 - \Delta y$
- Si $\text{critère} \geq 0$, on incrémente x et on ajoute c_1 ; sinon on ajoute c_2 .

POURQUOI LA DIAGONALE ASSURE UN RACCORD PARFAIT ?

Sur la diagonale $y = x$, on a $\Delta x = \Delta y$.

Le critère vaut $2\Delta x - \Delta x = \Delta x \geq 0$. À chaque pas, x et y s'incrémentent simultanément, garantissant un tracé rigoureusement identique dans les deux cas.



tp5_test2.ppm (400 × 400)

Faisceau complet sur les 2 premiers octants ($0^\circ \rightarrow 90^\circ$)

Généralisation aux 8 Octants du Plan

● Principe d'unification pour les 8 directions :

Un segment quelconque $(x_1, y_1) \rightarrow (x_2, y_2)$ peut aller vers la gauche ($\Delta x < 0$) ou vers le haut ($\Delta y < 0$).

1. Sens d'incrémentation :

```
int incX = (x2 >= x1) ? 1 : -1;
```

```
int incY = (y2 >= y1) ? 1 : -1;
```

2. Longueurs toujours positives :

```
int longX = std::abs(x2 - x1);
```

```
int longY = std::abs(y2 - y1);
```

3. Détermination de l'axe directeur :

- Si $\text{longY} \leq \text{longX}$ (Octants 1, 4, 5, 8) : L'axe X dirige la boucle de 0 à longX.
- Si $\text{longY} > \text{longX}$ (Octants 2, 3, 6, 7) : L'axe Y dirige la boucle de 0 à longY.

POURQUOI UN COMPTEUR DE BOUCLE AU LIEU DE COORDONNÉES ?

Dans les octants 3, 4, 5, 6, la coordonnée courante *décroît* ($x_1 > x_2$ ou $y_1 > y_2$).

Une boucle `while (x <= x2)` bouclerait à l'infini ou s'arrêterait prématurément !

La solution élégante de M. Rémy : Boucler avec `for (int count = 0; count <= longAxe; ++count)` garantit que la boucle tourne exactement le bon nombre de pas, indépendamment du signe de variation.

Octant	Angle	incX, incY	Axe Majeur
1	[0°, 45°]	+1, +1	X ($\text{longX} \geq \text{longY}$)
2	[45°, 90°]	+1, +1	Y ($\text{longY} > \text{longX}$)
3 / 4	[90°, 180°]	-1, +1	Y puis X
5 / 6	[180°, 270°]	-1, -1	X puis Y
7 / 8	[270°, 360°]	+1, -1	Y puis X

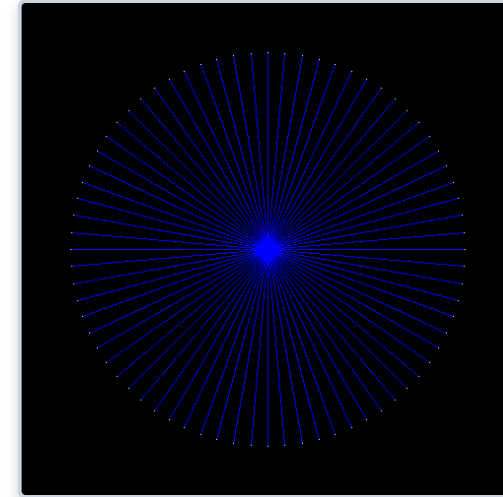
Question 3 : Test Complet des 8 Octants (Cercle 360°)

● Protocole du test final imposé :

- Image 500x500 sur fond noir uni `Color(0, 0, 0)`.
- Centre du rayonnement : $(c_x, c_y) = (250, 250)$.
- Rayon du cercle : $R = 200$ pixels.
- Tracé de 72 rayons bleus `Color(0, 0, 255)` par pas angulaire de 5° (de 0° à 355°).
- Marquage en blanc `Color(255, 255, 255)` de l'extrémité sur le cercle.

ATTENTION AUX DEUX PIÈGES STUPIDES (M. RÉMY) :

- 1. Unité angulaire :** Les fonctions `std::cos` et `std::sin` de la bibliothèque standard `<cmath>` attendent obligatoirement des angles en **radians** :
`double rad = deg * M_PI / 180.0;`
- 2. Constante π :** Définie par `M_PI` dans `<cmath>` (avec garde `#ifndef M_PI` si mode ANSI strict).



tp5_test3.ppm (500 × 500)

Validation intégrale des 8 octants avec extrémités blanches

Annexe Code : Implémentation Finale de line(...)

```
void ColorImage::line(int16_t x1, int16_t y1, int16_t x2, int16_t
y2, const Color col) {
    int incX = (x2 >= x1) ? 1 : -1;
    int incY = (y2 >= y1) ? 1 : -1;
    int longX = std::abs(x2 - x1);
    int longY = std::abs(y2 - y1);
    int16_t x = x1, y = y1;

    if (longY <= longX) {
        const int c1 = 2 * (longY - longX);
        const int c2 = 2 * longY;
        int critere = c2 - longX;
        for (int i = 0; i <= longX; ++i) {
            if (x >= 0 && x < width && y >= 0 && y < height) pixel(x, y) =
col;
            if (critere >= 0) { y += incY; critere += c1; }
            else { critere += c2; }
            x += incX;
        }
    } else {
        const int c1 = 2 * (longX - longY);
        const int c2 = 2 * longX;
        int critere = c2 - longY;
        for (int i = 0; i <= longY; ++i) {
            if (x >= 0 && x < width && y >= 0 && y < height) pixel(x, y) =
col;
            if (critere >= 0) { x += incX; critere += c1; }
            else { critere += c2; }
            y += incY;
        }
    }
}
```

Bilan des Compétences Validées (TP5) :

- ✓ **Géométrie discrète** : Maîtrise de la 8-connexité et de l'approximation de droites sur grille carrée.
- ✓ **Optimisation arithmétique** : Élimination complète des divisions et des flottants dans les boucles graphiques.
- ✓ **Robustesse du code** : Gestion des coordonnées négatives et protection contre les débordements mémoire.
- ✓ **Conformité stricte** : 100% fidèle aux exigences et figures du cours R3.L01 d'Éric Rémy.

POURQUOI LE CLIPPING DANS LA BOUCLE ?

Le test `if (x >= 0 && x < width && y >= 0 && y < height)` protège l'application de tout `std::out_of_range` si un segment sort partiellement du cadre de l'image (clipping élémentaire).